

Research Assistant Design and Build

1. Purpose and Scope

This Research Assistant is a small, policy-constrained assistant that can be run locally to quickly submit research requests, or deployed to a larger system to allow use throughout an organization. There is a simple front-end based in Plotly Dash to allow users to submit a research question. This question is validated by a set of defined restraints to prevent simple social engineering attacks, and then is passed to a FastAPI application running on Uvicorn. This FastAPI application coordinates the response by submitting API requests and passing data between Claude and Tavily (a public-web search tool), and then handles validation and report generation from the final response.

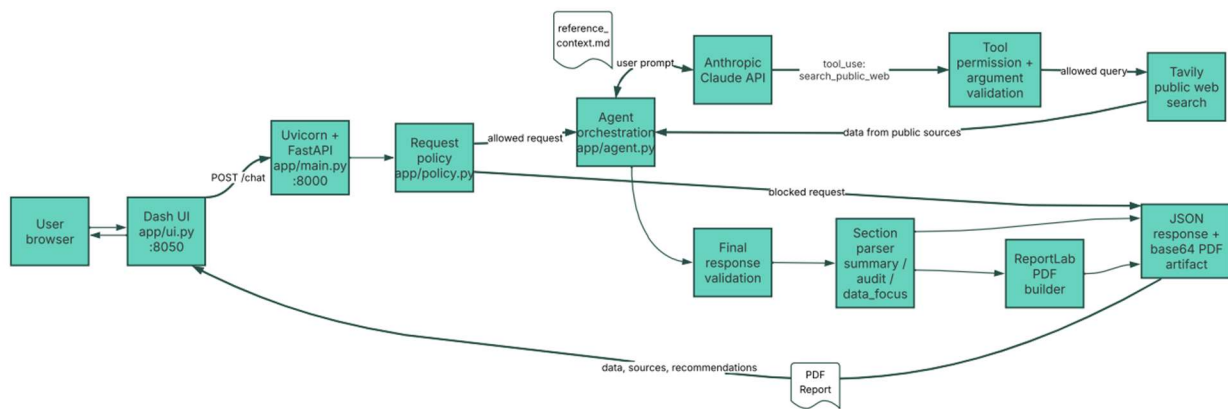
The application is designed for public search research and experiment design support. It does not claim that any experiment was performed, cannot perform any wet-lab actions, and only accesses public data sources. The result is entirely advisory: users must still review the data, citations, any uncertainties reported, and the final recommendations before relying on them.

The application is set up to allow running the assistant through Docker for reproducible and quick local development and internal use. It does not provide any authentication or secret-management services, as all secrets should be managed through the environment files.

2. Architecture

This Research Assistant is based on the following two user-facing processes running simultaneously:

- Uvicorn/FastAPI (app.main:app) listens on port 8000 with the standard setup and owns the API, policy gate, agent call, response validation, response parsing, and PDF report generation. This application handles all requests to Claude and Tavily, and validations of the prompts and responses.
- Plotly Dash (app.ui:app) listens on port 8050 with the standard setup and owns the user interface. This provides a simple web-based UI where a user can submit a research request, and the Dash server then calls the FastAPI /chat endpoint server-side to trigger the agent and render the different response formats within the UI.



The Docker image starts both processes in one container. The agent loop triggered by the FastAPI application is bounded. Each end-user request permits at most five Claude model iterations and ten Tavily public search calls to prevent overuse of tokens if an error is encountered. These limits can be configured in `agent.py:Agent.run()`. The agent cannot request any arbitrary URLs or unregistered tools, and any tool calls must pass application policy and argument validation.

In addition, there is a `reference_context.md` file that is passed as additional context to all Claude requests. This context file contains context for the agent, as well as limitations, requested response formats, and additional instructions for how to handle and process any data.

3. Request and Response Lifecycle

Request Path

1. The user enters a question in the Dash UI and selects Run research.
2. Dash sends `{"message": "..."} to POST /chat.`
3. FastAPI validates the request with Pydantic and applies `check_user_request()`.
4. The request policy rejects empty input, private or confidential-data requests, and common attempts to override restrictions.
5. `Agent.run()` combines the user message with `reference_context.md` and sends it to Claude with `SYSTEM_PROMPT` and the available tool definition.
6. If Claude requests `search_public_web`, the application checks the tool name and arguments before calling Tavily.
7. Search results are returned to the agent as structured tool results. Claude can use them to produce the final answer.
8. The final text is checked by `validate_final_response()`. The application also requires the three exact section headings for a structured answer.

9. FastAPI parses the headings into three independent fields: summary, audit, and data_focus.
10. ReportLab creates a PDF from those fields. The PDF is returned as a base64-encoded download artifact.
11. Dash renders the three fields in the Summary, Technical / Audit, and Data Focus tabs and exposes the PDF as a download link.

4. Instructions and Control Boundaries

The application has multiple instruction layers to provide restrictions with differing authorities.

- Application policy
 - app/policy.py is the policy boundary that defines the public-source requirements, forbids external actions or fabricated evidence, and requires citation. It also validates user input and restricts to only authorized tools (Claude and Tavily). User messages and model outputs cannot override these checks.
- System prompt
 - SYSTEM_PROMPT in app/agent.py is a static prompt that describes the assistant's role and expected answer quality. It has explicit instructions to distinguish fact from reasoning, cite all sources used, and avoid fabricating or hallucinating any evidence.
- Reference context
 - app/reference_context.md contains domain and formatting guidance, such as formatting requirements, source citation expectations, handling of missing evidence, and default ranking criteria if none is provided in the user prompt. The agent appends this context to the user prompt, but does not override the system prompt or application policy.